

Security Operation Centre (SOC) Log

Network Incident & Threat Analysis Report

Analyst: Rozina Khatun

Date: 07-07-2026

Executive Summary

Incident Overview: During tactical network traffic analysis, the Suricata Intrusion Detection System (IDS) flags multiple internal-to-external unencrypted HTTP network connections. These were originated from the internal enterprise baseline environment (\$HOME_NET).

The scope of this deployment was to systematically validate the detection runtime environment, test baseline signature ingestion, and map structural sessions. While standard outbound HTTP GET requests form part of normal web browsing telemetry, the continuous existence of unencrypted, clear text application traffic presents an operational policy violation, potential compliance vulnerability, or vector for command-and-control (C2C) communication channel tracking.

Risk Rating Assessment: Low (Operational Review Required)

The operational risk vector is classified as low because the activities are encapsulated within a verified sandbox lab framework. However, the technical mechanics map exactly to real-world data exfiltration vulnerabilities, prompting an immediate requirement for rule tuning and secure protocol migration paths.

Technical analysis & Evidence

A. Signature Identification and Structure Diagnostics

The analysis focused on structural features of an ingested custom rule inside the **custom.rules** configuration tree. The engineering framework of the rule breaks down into three distinct building blocks: an **Action statement**, a **Network Header** directive, and **custom Option** parameters.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"GET on wire"; flow:established,to_server; content:"GET"; http_method; sid:12345; rev:3;)
analyst@07dbf040f06b:~$ ls -l /var/log/suricata
```

- **Action Strategy:** The alert directive sets up a tracking pipeline to flag matching events to the system logs immediately.

- **Network Header Definition:** The scope “*http \$HOME_NET any -> \$EXTERNAL_NET any*” bounds the rule mechanics explicitly to look for **HTTP** application traffic initiating from internal assets running on any source port heading outwards to external nodes.
- **Payload Matching Mechanics:** The parameter **content:”Get” ; http_method;** option instructs the packet analysis engine to inspect the specific **HTTP** header fields for the raw string value of a client request structure.
- **Metadata Fields:** Uniquely registered as **Signature ID (SID)** 1234 under Revision string 3.

B. Pipeline Execution & Alerts Engineering

To run the engine simulation, the internal validation structure required processing the raw source file ‘*sample.pcap*’ while bypassing hardware-level check validation configurations to avoid frame dropping issues:

```
analyst@97dbf040f96b:~$ sudo suricata -r sample.pcap -S custome.rules -k none
```

Upon execution, the file system population verification run against the logging tree directory path ‘*/var/log/suricata/*’ verified successful alerts tracking generation inside the core ‘*fast.log*’ file system layer. Every log event clearly captures a single sequential timestamp block, signature details, and the core communication endpoint matrices.

C. Advanced Data Slicing & Session Mapping via JSON Log Parsing

Because raw telemetry entries inside “*eve.json*” contain wide structural data arrays, advanced parsing commands were executed utilising the “*jq*” processing ecosystem.

Step 1: Visual Structural Normalisation

To establish schema readability, interactive text parsing and pagination was orchestrated over the core data interface stream:

```
analyst@97dbf040f96b:~$ jq . /var/log/suricata/eve.json | less
```

Step 2. Targeted Key-Value Attribute Aggregation

High-value operational network metadata attributes were filtered directly into condensed programmatic arrays. This eliminated superfluous logs clutter:

```
analyst@350c67bd479c:~$ jq -c "[.timestamp, .flow_id, .alert.signature, .proto, .dest_ip]" /var/log/suricata/eve.json
```

Step 3: Stateful Session Reconstruction Tracking

To accurately cross-examine the total conversation lifestyles and analyse packet flows, the unique tracking string “**flow_id**” discovered during programmatic extraction, was isolated and queried:

```
analyst@bd31efce953c:~$ jq "select(.flow_id==1169891449796757)" /var/log/suricata/eve.json
```

Isolating this key enabled full state tracking reconstruction of the associated TCP handshake patterns, payload exchange phases, and session breakdown flags across the log ledger.

Lessons Learned & Post-Incident Review

This operational exercise taught me a lot about tuning rules and managing logs. First, the custom rule I tested, is way too broad for a real network. It alerts on every single HTTP GET request, which would cause massive alert fatigue for a real SOC team. In the future, rules need to be much more specific, targeting things like malicious User-Agents or bad domains.

On the logging side, I learned that raw JSON files are a nightmare to read manually during an active incident. However, using “jq” to filter by “flow-id” completely changed the look. It made it incredibly easy to piece the whole network conversation back together. Finally, seeing cleartext HTTP traffic on Port 80 in a reminder that organisations need to enforce HTTPS across the board to protect data from sniffing.

Document Authorisation

This technical document serves as a complete validation record of incident analysis capabilities. The workflows outlined represent standard defensive methodologies for deployment verification within an enterprise security architecture framework.

Appendix

Scenario

In this lab, I am a security analyst who must monitor traffic on my employer's network. I'll be required to configure Suricata and use it to trigger alerts. I have been given a sample pcap file.

Here is how I have done it:

Task 1. Examine a custom rule in Suricata

To do this, I ran following commands:

- `/home/analyst`
- `/var/log/suricata` directory to locate files
- `ls` to bring the file list
- `cat custom.rules` to display the rule in the `custom.rules` file

```
analyst@97dbf040f96b:~$ ls
custom.rules  sample.pcap
analyst@97dbf040f96b:~$ cat custom.rules
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"GET on wire"; flow:established,to_server; content:"GET"; http_method; sid:12345; rev:3;)
analyst@97dbf040f96b:~$ ls -l /var/log/suricata
total 0
analyst@97dbf040f96b:~$ sudo suricata -r sample.pcap -S custome.rules -k none
15/2/2026 -- 08:15:05 - <Notice> - This is Suricata version 6.0.1 RELEASE running in USER mode
15/2/2026 -- 08:15:06 - <Warning> - [ERRCODE: SC_ERR_NO_RULES(42)] - No rule files match the pattern custome.rules
15/2/2026 -- 08:15:06 - <Warning> - [ERRCODE: SC_ERR_NO_RULES_LOADED(43)] - 1 rule files specified, but no rules were loaded!
15/2/2026 -- 08:15:06 - <Notice> - all 2 packet processing threads, 4 management threads initialized, engine started.
15/2/2026 -- 08:15:06 - <Notice> - Signal Received. Stopping engine.
15/2/2026 -- 08:15:06 - <Notice> - Pcap-file module read 1 files, 200 packets, 54238 bytes
analyst@97dbf040f96b:~$ ls -l /var/log/suricata
total 8
-rw-r--r-- 1 root root 0 Feb 15 08:15 eve.json
-rw-r--r-- 1 root root 0 Feb 15 08:15 fast.log
-rw-r--r-- 1 root root 2768 Feb 15 08:15 stats.log
-rw-r--r-- 1 root root 1652 Feb 15 08:15 suricata.log
analyst@97dbf040f96b:~$
```

Here, the rules consist of three components: an action, a header and rule options. The breakdown is -

- **alert** bit is an **action**
- **http \$HOME_NET any -> \$EXTERNAL_NET any** is a **header** (has protocol, source and destination IP addresses, source and destination ports and traffic direction).

- **msg** bit provides the alert text: **Get on wire**.
- The **flow:established, to_server** option determines that the packets from the client to the server should be matched.
- **content:"Get"** option tell Suricata to look for the word **GET** in the content of the **http.method** portion of the packet.
- **sid:12345** is the **signature ID** that defines the rule.
- The **rev:3** bit is revision version 3.

To summarise, this signature triggers an alert whenever Suricata observes the text **GET** as the HTTP method in an HTTP packet from the home network going to the external network.

Task 2. Trigger a custom rule in Suricata

I ran command **ls -l /var/log/suricata** to list the files. Here, I can see that there is no file in this directory at his point. Next, I ran **sudo suricata -r sample.pcap -S custom.rules -k none** where **-r sample.pcap** specifies an input file to mimic network traffic, **-S custom.rules** tells Suricata to use rules defined in that file and **-k none** instructs Suricata to disable all checksum checks. Next, I ran the command **ls -l /var/log/suricata** again to list the files in /var/log/suricata folder.

After that, I used **cat** to display the **fast.log** file. Each line or entry in the **fast.log** file corresponds to an alert generated by Suricata when it processes a packet that meets the conditions of an alert generating rule. Each alert line includes the message that identifies the rule that triggered the alert, as well as the source, destination, and direction of the traffic.

```
analyst@97dbf040f96b:~$ cat /var/log/suricata/fast.log
11/23/2022-12:38:34.624866  [**] [1:12345:3] GET on wire [**] [Classification: (null)] [Priority: 3] {TCP} 172.21.224.2:49652 -> 142.250.1.139:80
11/23/2022-12:38:58.958203  [**] [1:12345:3] GET on wire [**] [Classification: (null)] [Priority: 3] {TCP} 172.21.224.2:58494 -> 142.250.1.102:80
analyst@97dbf040f96b:~$
```

Task 3. Examine eve.json output

I have used **cat /var/log/suricata/eve.json** command to display the entries in that file.

```

analyst@97dbf040f96b:~$ cat /var/log/suricata/eve.json
{"timestamp":"2022-11-23T12:38:34.624866+0000","flow_id":193185821915285
,"pcap_cnt":70,"event_type":"alert","src_ip":"172.21.224.2","src_port":4
9652,"dest_ip":"142.250.1.139","dest_port":80,"proto":"TCP","tx_id":0,"a
lert":{"action":"allowed","gid":1,"signature_id":12345,"rev":3,"signatur
e":"GET on wire","category":"","severity":3},"http":{"hostname":"opensou
rce.google.com","url":"/","http_user_agent":"curl/7.74.0","http_content_
type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":301
,"redirect":"https://opensource.google/","length":223},"app_proto":"http
","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver":357,"byte
s_toclient":788,"start":"2022-11-23T12:38:34.620693+0000"}}
{"timestamp":"2022-11-23T12:38:58.958203+0000","flow_id":169963486514917
2,"pcap_cnt":151,"event_type":"alert","src_ip":"172.21.224.2","src_port"
:58494,"dest_ip":"142.250.1.102","dest_port":80,"proto":"TCP","tx_id":0,
"alert":{"action":"allowed","gid":1,"signature_id":12345,"rev":3,"signat
ure":"GET on wire","category":"","severity":3},"http":{"hostname":"opens
ource.google.com","url":"/","http_user_agent":"curl/7.74.0","http_conten
t_type":"text/html","http_method":"GET","protocol":"HTTP/1.1","status":3
01,"redirect":"https://opensource.google/","length":223},"app_proto":"ht
tp","flow":{"pkts_toserver":4,"pkts_toclient":3,"bytes_toserver":357,"by
tes_toclient":797,"start":"2022-11-23T12:38:58.955636+0000"}}
analyst@97dbf040f96b:~$

```

The output here is raw content of that eve.json file. So, I ran command **jq**.
/var/log/suricata/eve.json | less to display entries in an improved format.

```

analyst@97dbf040f96b:~$ jq . /var/log/suricata/eve.json | less
{
  "timestamp": "2022-11-23T12:38:34.624866+0000",
  "flow_id": 193185821915285,
  "pcap_cnt": 70,
  "event_type": "alert",
  "src_ip": "172.21.224.2",
  "src_port": 49652,
  "dest_ip": "142.250.1.139",
  "dest_port": 80,
  "proto": "TCP",
  "tx_id": 0,
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 12345,
    "rev": 3,
    "signature": "GET on wire",
    "category": "",
    "severity": 3
  },
  "http": {
    "hostname": "opensource.google.com",
    "url": "/",
    "http_user_agent": "curl/7.74.0",
    "http_content_type": "text/html",
    "http_method": "GET",
    "protocol": "HTTP/1.1",

```

Next, I used `jq -c "[.timestamp,.flow_id,.alert.signature,.proto,.dest_ip]" /var/log/suricata/eve.json` command to extract specific event data (specified in the square brackets) from that eve.json file:

```
analyst@350c67bd479c:~$ jq -c "[.timestamp,.flow_id,.alert.signature,.proto,.dest_ip]" /var/log/suricata/eve.json
parse error: Invalid numeric literal at line 2, column 0
analyst@350c67bd479c:~$ jq -c "[.timestamp,.flow_id,.alert.signature,.proto,.dest_ip]" /var/log/suricata/eve.json
["2022-11-23T12:38:34.624866+0000",2193111573493909,"GET on wire","TCP","142.250.1.139"]
["2022-11-23T12:38:58.958203+0000",542577118057716,"GET on wire","TCP","142.250.1.102"]
analyst@350c67bd479c:~$ jq "select(.flow_id==X)" /var/log/suricata/eve.json
jq: error: X/0 is not defined at <top-level>, line 1:
select(.flow_id==X)
jq: 1 compile error
analyst@350c67bd479c:~$
```

Finally, as part of the task, I have used `jq` command to display all event logs related to a specific `flow_id` from the `eve.json` file. The `flow_id` is very useful as it correlates network traffic that belong to the same network flows. By this, I finished my lab as I configured Suricata to trigger alerts on network traffic.

```
analyst@bd31efce953c:~$ jq "select(.flow_id==1169891449796757)" /var/log/suricata/eve.json
{
  "timestamp": "2022-11-23T12:38:34.624866+0000",
  "flow_id": 1169891449796757,
  "pcap_cnt": 70,
  "event_type": "alert",
  "src_ip": "172.21.224.2",
  "src_port": 49652,
  "dest_ip": "142.250.1.139",
  "dest_port": 80,
  "proto": "TCP",
  "tx_id": 0,
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 12345,
    "rev": 3,
    "signature": "GET on wire",
    "category": "",
    "severity": 3
  },
  "http": {
    "hostname": "opensource.google.com",
    "url": "/",
    "http_user_agent": "curl/7.74.0",
    "http_content_type": "text/html",
  }
}
```